Preliminary Results on Evaluating Large Language Models for Labeling Root Cause Categories of Fixed Flaky Tests

Yang Chen
University of Illinois Urbana-Champaign
Urbana, IL, USA
yangc9@illinois.edu

Kaiyao Ke
University of California, Berkeley
Berkeley, CA, USA
kaiyaoke@berkeley.edu

Darko Marinov
University of Illinois Urbana-Champaign
Urbana, IL, USA
marinov@illinois.edu

Abstract

Flaky tests are an important hindrance in practical software development. Researchers have developed many automated techniques for detecting, mitigating, and fixing flaky tests. Such new techniques are often inspired by studying past fixed flaky tests. Several papers present categorizations of flaky tests. However, such categorizations were obtained mostly manually, by researchers reading the fixes (code changes, commit messages, pull request discussions, etc.) and grouping the tests based on their likely root cause.

In this paper, we present our preliminary results on evaluating the use of large language models (LLMs) for labeling categories of fixed flaky tests. We evaluate two state-of-the-art models (GPT-4 and GPT-5) on two datasets of fixed flaky tests: one dataset with 52 tests released last year and one newly introduced dataset with 244 tests. We prompt LLMs to label each test into one of the 13 categories. The preliminary results show several interesting results: (1) LLMs achieve accuracies ranging from 38% to 59%, which is not very high, but note that it is still much better than random guess because we do not consider only binary classification (whether a test is flaky or not); (2) GPT-5 performs better than GPT-4 (as expected); and (3) the overall accuracy is similar on both datasets (showing that data contamination is not an issue, at least not yet).

Keywords

Flaky Tests, Root Cause Categorization, Large Language Models

ACM Reference Format:
Yang Chen, Kaiyao Ke, and Darko Marinov. 2026. Preliminary Results on Evaluating Large Language Models for Labeling Root Cause Categories of Fixed Flaky Tests. In *3rd International Flaky Tests Workshop 2026 (FTW '26), April 12–18, 2026, Rio de Janeiro, Brazil.* ACM, New York, NY, USA, 2 pages. https://doi.org/10.1145/3786178.3788329

1 Motivation

Prior work on labeling flaky tests using machine learning and LLMs [2, 3, 5–7, 10, 11] has mostly focused on determining whether a test is flaky or not. Much less attention has been devoted to determining the category of flakiness (the root cause of flakiness) [1, 4, 5, 8, 11]. Development teams can benefit from knowing the flakiness category, e.g., to make an informed decision whether to rerun a failure, prioritize which tests to fix, or assign the fixing to a specific developer on the team. However, labeling unfixed tests just based on the failure (e.g., stack trace), the test code, and the code under test is a rather challenging problem that the current LLMs struggle with. Therefore, in this paper, we consider a related problem of labeling *already fixed* tests – while such labeling does not directly help developers, it does *help researchers* who study historical fixes to advance tools for detecting, mitigating, and fixing flaky tests.



This work is licensed under a Creative Commons Attribution 4.0 International License.
FTW '26, Rio de Janeiro, Brazil
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2409-1/26/04
https://doi.org/10.1145/3786178.3788329

2 Approach

Two Datasets: We use two datasets referred to as *RustFT* and *MPFT*. RustFT was released by Schroeder et al. [9]. It contains 52 fixed flaky tests written in the Rust programming language. Three researchers (with experience in studying flaky tests) have labeled each of those tests into one of nine categories: Environment, Logic, Async wait, Concurrency, Network, Time, I/O, Randomness, and Unordered collections. Since the dataset has been public (https://github.com/jts13/RustFlakyTest) for several months, it could have been used in training or finetuning the recent LLMs.

MPFT is a new dataset introduced in this paper. It contains 244 fixed flaky tests written in 17 programming languages (note"MP" in the name). 36 students (with limited experience in studying flaky tests) have labeled each of those tests into one of the 11 categories: seven categories identical to those in RustFT (Async wait, Concurrency, Network, Time, I/O, Randomness, and Unordered collections) and four additional ones: Test order dependency, OS, Floating point operations, and Resource leak. Except for the OS category, the other ten are defined by Luo et al. [5].

One Category per Test: Our task asks to label each fixed flaky test into exactly one category. For many tests, such labeling is clear and most experienced developers would agree, but for some tests, the labeling is rather challenging. For example, the original Luo et al. [5] paper labeled 27 out of 201 (13.4%) tests as "hard to label". Similarly, 11 out of 77 (14.2%) tests in Android [11] are marked as "hard to classify". Moreover, some tests may have root causes that come from two or more categories.

Following prior work [5], we label each test into precisely one category, which appears the most dominant for the root cause, and label each test by two (or more) researchers to ensure correctness. In case of disagreement, we use the label provided by the researcher with the most experience in flaky tests. (This decision is for pragmatic reasons as we could not easily communicate among the students from the class and researchers on the project to resolve the differences via a dialogue.) As a result, we find some disagreement with the labels released in RustFT.

Four Kinds of Prompts: We evaluate four kinds of prompts that combine two aspects: (1) provide just the category names or also definitions —we call these *NoDef* and *Def*; and (2) provide code patch or only the rest of the fixes—we call these *Patch* and *NoPatch*. Figure 1 shows the overall prompt structure.

NoDef provides only the names of the root cause categories, without providing their definitions. Def additionally provides their

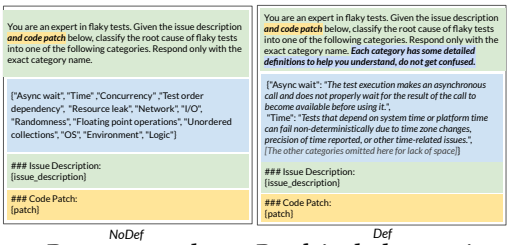


Figure 1: Prompt templates; *Patch* includes text in yellow

definitions, directly copied from Luo et al. [5] or Schroeder et al. [9], with one difference to add more description for the Concurrency category that some testing frameworks run multiple tests in parallel. The definitions help models better understand the root causes and reduce ambiguity. The definitions are not overly long and fit into the context window size (128k) for both GPT-4 and GPT-5. Across most settings, both models show performance improvements, confirming that they benefit from the additional definitions.

The Patch variant includes the code patch; NoPatch omits it. Including patches in prompts can provide models with more context to understand the problem. The NoPatch setting somewhat simulates scenarios where the issue has not yet been fixed, toward real-world situations in which developers diagnose flaky tests and reason about their causes before implementing a patch.

Two Models: We use two state-of-the-art and cost-effective models: GPT-4 (turbo version) and GPT-5. For each model, we evaluate all four kinds of prompts using the default temperature of 1.0.

3 Evaluation

Table 1 shows the results for all 16 combinations of datasets, models, and prompts in our experiments. Comparing between two models, GPT-5 consistently outperforms GPT-4 in every configuration; GPT-5's stronger reasoning capability to understand both natural language and source code leads to more accurate category predictions. The performance gap between the two models is smaller on the RustFT dataset than on MPFT, possibly because the smaller number of instances constrains large variations. Interestingly, GPT-4 demonstrates a consistent precision rate on RustFT (25 out of 52 tests), although the exact correctly predicted instances can differ.

Comparing between NoDef and Def, we observe that adding detailed definitions of root causes helps models understand the context and reduces potential ambiguity in inference, leading to an average performance improvement ranging from 0% to 8.7%.

Comparing between NoPatch and Patch, we find that including code changes helps improve model performance with an average increase of 0% to 21.2%. This improvement is expected as PR descriptions and Git commit messages alone may lack sufficient details to reveal the exact cause of flakiness, whereas code changes provide concrete code that helps models reason more effectively.

To further understand why LLMs fail to identify the root causes of flakiness, we manually examined all instances where at least one model failed under the Patch setting (i.e., with patches included in the prompt). We did not choose the NoPatch setting, as it might not fully reflect the models' capabilities due to the absence of code context. As mentioned earlier, some PR descriptions consist of only a brief sentence, such as "Fix a flaky test", lacking sufficient information, which can limit the models' ability to reason about the actual flakiness. We identified three main reasons for model failures (and show in parentheses the number of mispredictions across both datasets for the best combination: GPT-5, Def, Patch).

Table 1: Correct model predictions across all settings

Dataset	RustFT (Total #Tests: 52)				MPFT (Total #Tests: 244)			
Model	GPT-4		GPT-5		GPT-4		GPT-5	
Prompt	NoDef	Def	NoDef	Def	NoDef	Def	NoDef	Def
NoPatch	25	25	25	26	94	99	122	136
Patch	25	25	37	38	113	120	142	151
Avg.	**25.0**	**25.0**	**31.0**	**32.0**	**103.5**	**109.5**	**132**	**143.5**

R1 (82 mispredictions). Misunderstanding the root cause. For example, when a patch increases timeout values to mitigate a flaky test caused by async wait, the model focuses only on the timing aspect and incorrectly categorizes the root cause as Time instead of Async wait. Another example is a Concurrency test where multiple threads use a mocked file download (flakiness is due to lock races across threads rather than network instability); the LLM mislabels this test as Network, because terms such as "requests", "client", and "downloader" resemble actual network activity.

R2 (18 mispredictions). LLM misunderstanding the definition of flaky test categories. Although Def provides detailed definitions for each category, the LLM may still struggle to fully understand or differentiate them, particularly in borderline cases. For example, Luo et al. [5] define Concurrency as nondeterminism caused by undesirable interactions between threads but excluding asynchronous calls covered by the Async wait category. In our ground truth, we label tests as Async wait if flakiness arises from multiple asynchronous calls without proper synchronization; however, the LLM often predicts Concurrency in such cases, likely because it focuses on the multithreaded aspect of the test.

R3 (7 mispredictions). Insufficient code context. For example, some tests are caused by unordered collections defined in external libraries that are not in the modified files, so models lack sufficient context to understand the underlying cause of flakiness, leading to mispredictions. In other cases, the PR associated with the code patch lacks a description, and the code diff itself provides insufficient context to understand the rationale behind the fix.

4 Conclusion

We presented a preliminary evaluation of LLMs for labeling the root causes of fixed flaky tests. Across two datasets (RustFT and MPFT) and four prompt settings, GPT-5 consistently outperformed GPT-4. Adding category definitions and code patches improves accuracy by providing richer context. We analyzed mispredictions to understand limitations and plan to extend the study to unfixed flaky tests in the future.

References

[1] Amal Akli, Guillaume Haben, Sarra Habchi, Mike Papadakis, and Yves Le Traon. 2023. Predicting Flaky Tests Categories using Few-Shot Learning. In *AST*.
[2] Abdulrahman Alshammari, Christopher Morris, Michael Hilton, and Jonathan Bell. 2021. FlakeFlagger: Predicting Flakiness Without Rerunning Tests. In *ICSE*.
[3] Sakina Fatima, Taher A. Ghaleb, and Lionel Briand. 2023. Flakify: A Black-Box, Language Model-Based Predictor for Flaky Tests. *TSE* (2023).
[4] Sakina Fatima, Hadi Hemmati, and Lionel C. Briand. 2024. FlakyFix: Using LLMs for Predicting Flaky Test Fix Categories and Test Code Repair. *TSE* (2024).
[5] Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. 2014. An Empirical Analysis of Flaky Tests. In *FSE*.
[6] Owain Parry, Gregory M. Kapfhammer, Michael Hilton, and Phil McMinn. 2021. A Survey of Flaky Tests. *TOSEM* (2021).
[7] Shanto Rahman, Abdelrahman Baz, Sasa Misailovic, and August Shi. 2024. Quantizing Large-Language Models for Predicting Flaky Tests. In *ICST*.
[8] Shanto Rahman, Saikat Dutta, and August Shi. 2025. Understanding and Improving Flaky Test Classification. In *OOPSLA*.
[9] Tom Schroeder, Minh Phan, and Yang Chen. 2025. A Preliminary Study of Fixed Flaky Tests in Rust Projects on GitHub. In *FTW*.
[10] A. Tahir, S. Rasheed, J. Dietrich, N. Hashemi, and L. Zhang. 2023. Test Flakiness' Causes, Detection, Impact and Responses: A Multivocal Review. *JSS* (2023).
[11] Swapna Thorve, Chandani Sreshtha, and Na Meng. 2018. An Empirical Study of Flaky Tests in Android Apps. In *ICSME*.